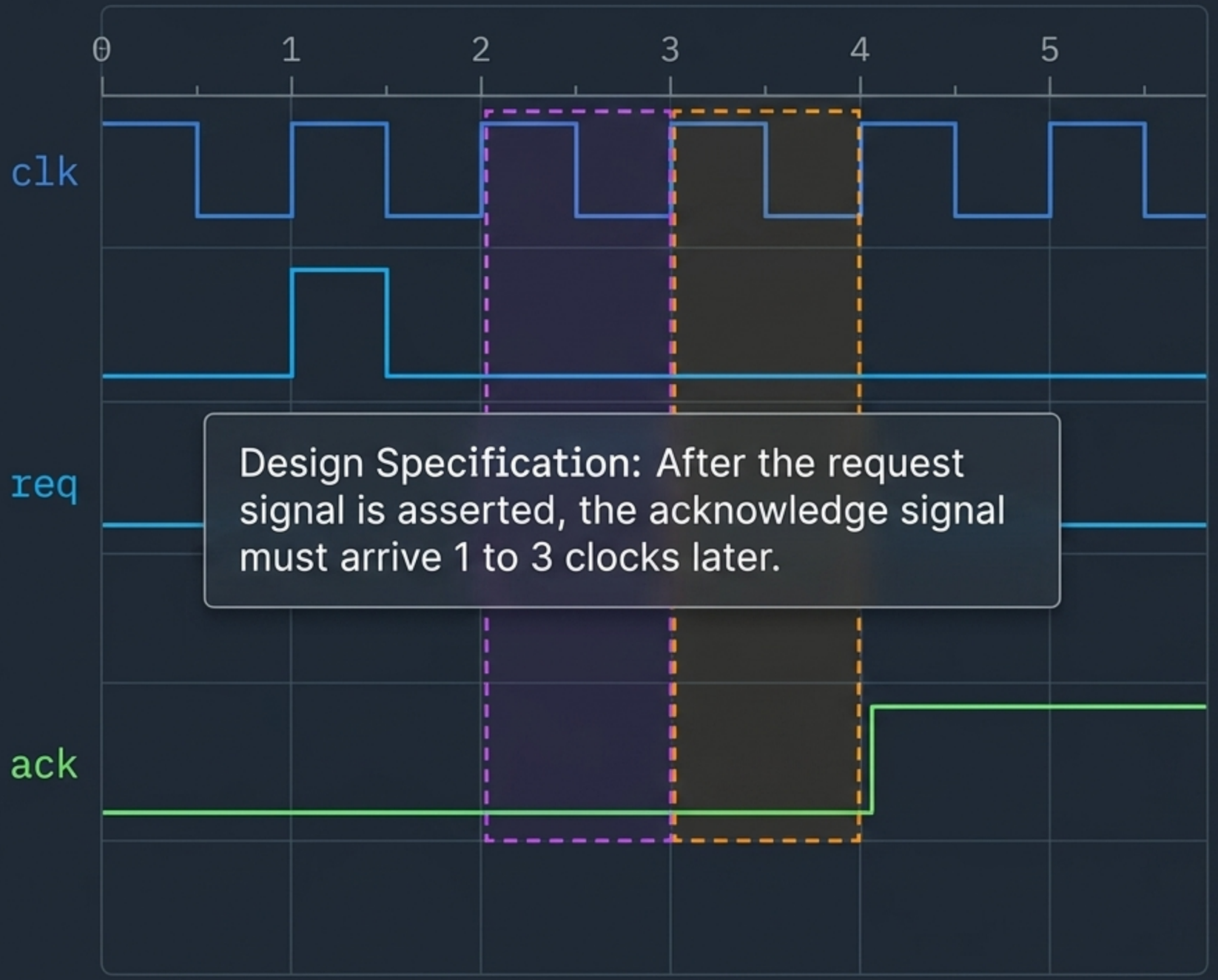


The Architecture of SystemVerilog Assertions

A Definitive Explainer and Syntax Desk Reference

NotebookLM



Why we use SVA:

Validate Behavior

Checks specific conditions or sequences over simulation time.

Document Intent

Translates English specs into executable code.

Localize Failures

Catches bugs at the source, not just at the final output.

Formal Semantics

Provides structural bounds for formal verification tools.

NotebookLM

The Assertion Taxonomy Matrix

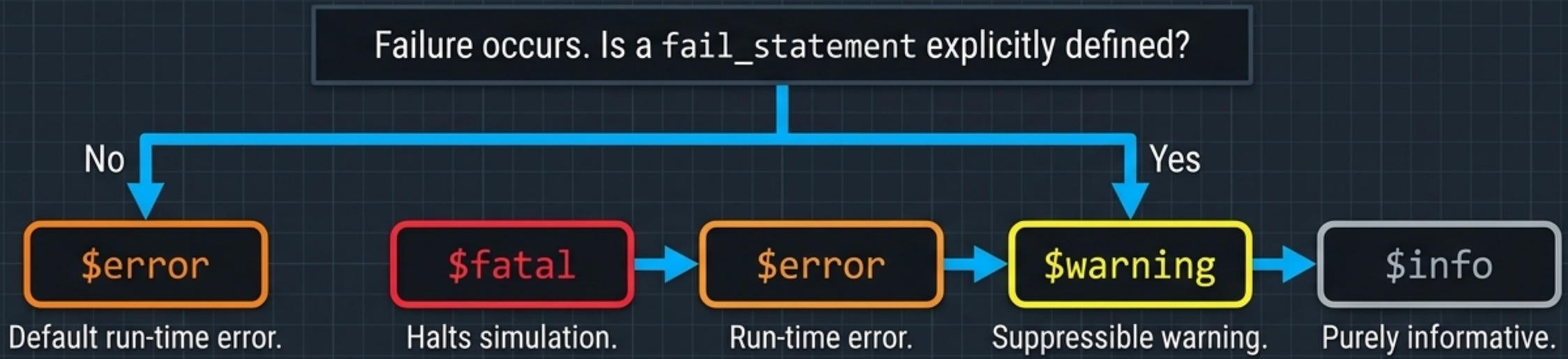
	Immediate	Concurrent
Evaluation Time	Current simulation time (0-delay).	Spread over multiple clock cycles.
Clock Dependency	Unlinked to clocks (acts like an if..else statement).	Evaluated strictly at the occurrence of a clock tick.
Placement	Procedural blocks only.	Procedural blocks, modules, interfaces, or programs.
Key Differentiator	Evaluates standard boolean expressions. Syntax: `label: assert(expression) action_block;`	Evaluates properties over time (uses the property keyword). Syntax: `c_assert: assert property(@(posedge clk) not(a && b));`

NotebookLM

IMMEDIATE ASSERTION ANATOMY



FAILURE SEVERITY FLOWCHART



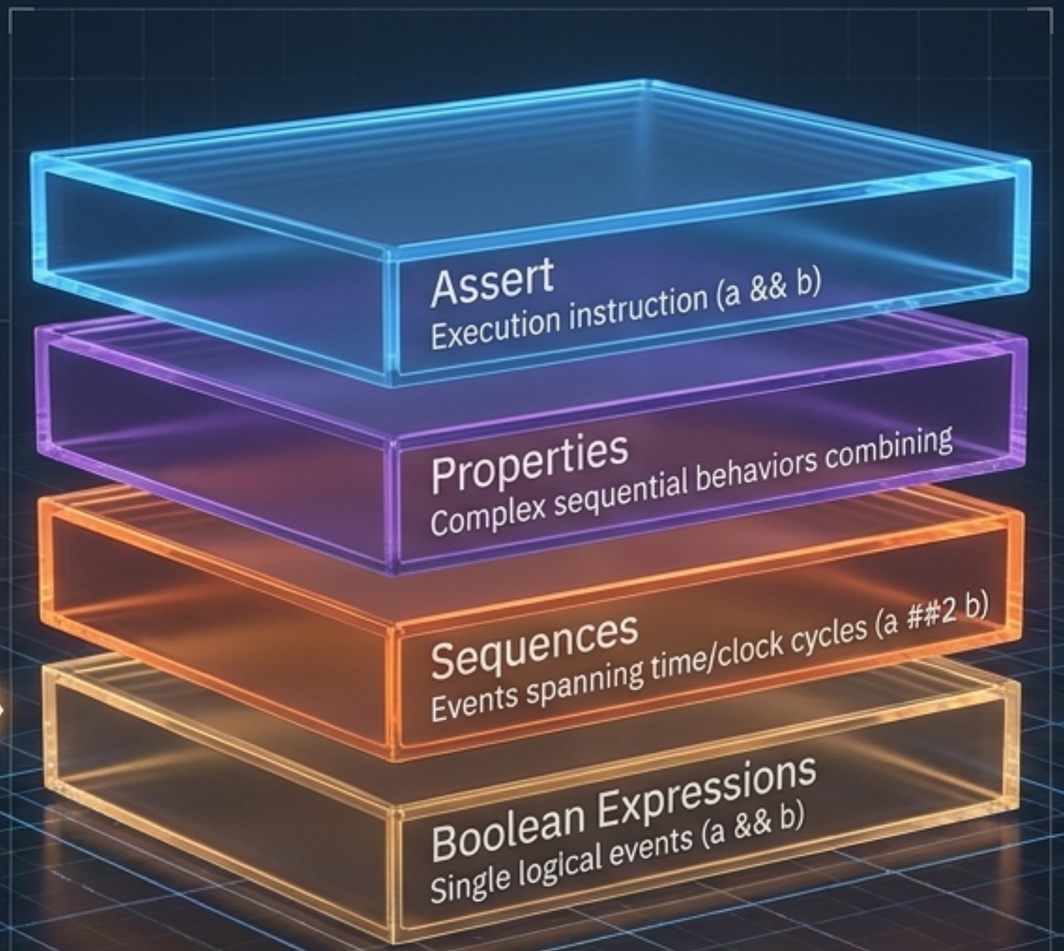
NotebookLM

The Bottom-Up Architecture

```
/* SystemVerilog */

start properties
    waveform in ct (a)
    waveform in nual && b);
end propperties;
```

You Are Here



Assert
Execution instruction to actively verify the property

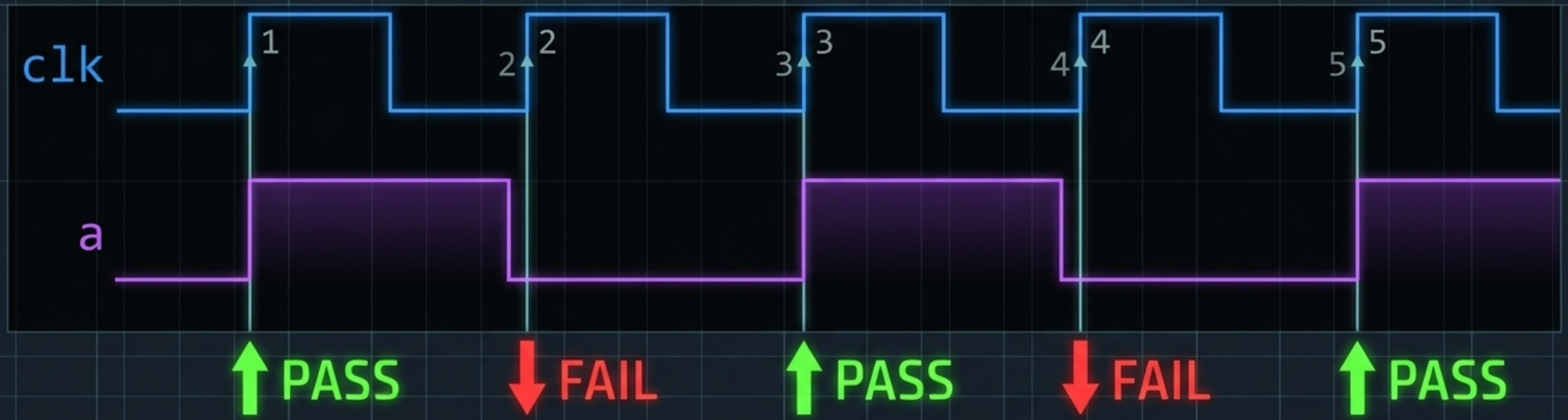
Properties
Complex sequential behaviors combining sequences logically

Sequences
Events spanning time/clock cycles (a ##2 b)

Boolean Expressions
Single logical events (a && b)

NotebookLM

```
sequence seq_1;
  @(posedge clk) a==1;
endsequence
```

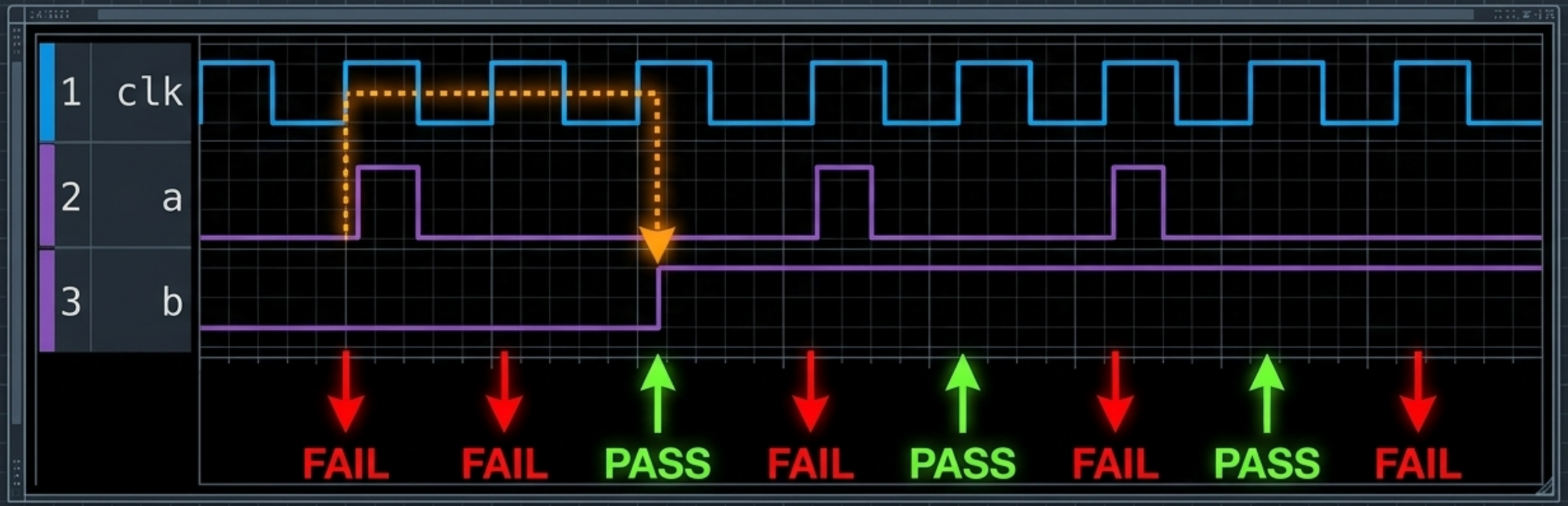


Sequences evaluate at the exact moment of the clock tick based on the sampled variable values.

NotebookLM

The Temporal Anchor

```
sequence seq;
  @(posedge clk) a ##2 b;
endsequence
```



NotebookLM

The Timing Window Spectrum

Fixed Delay

`##2`



Exact clock cycles.

Bounded Window

`##[1:4]`



Signal must be high at least once within the specified window.

Indefinite Window (Eventuality)

`##[1:$]`

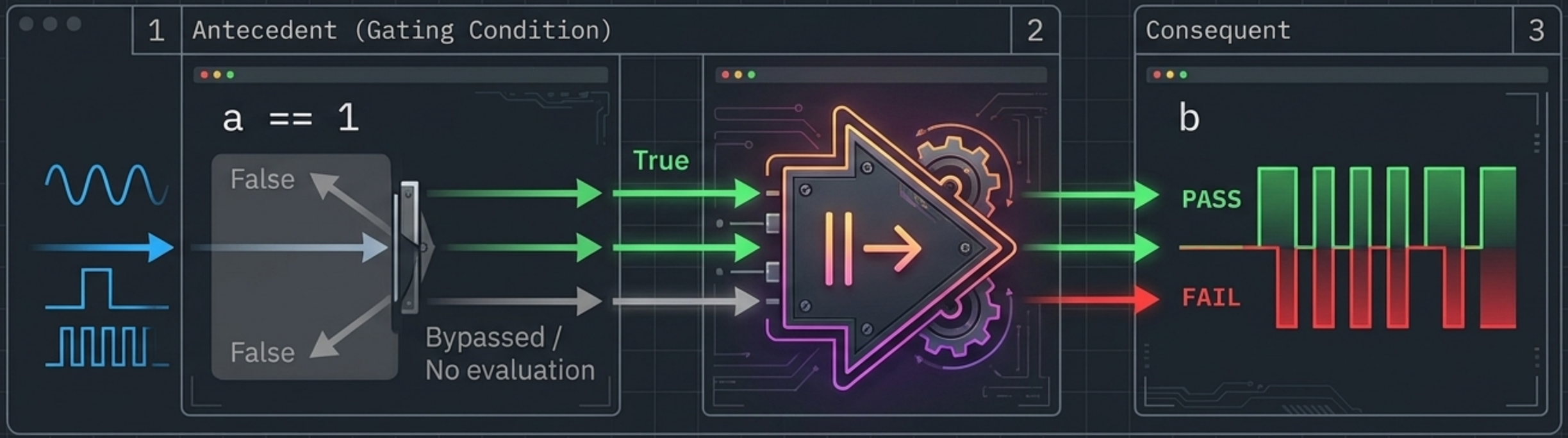


The Eventuality Operator. The checker continues searching until the end of simulation.

NotebookLM

The Antecedent Gate

⚠ If “a” is not high on every clock edge, the sequence issues an error.
But we only care about checking “b” when “a” is high.

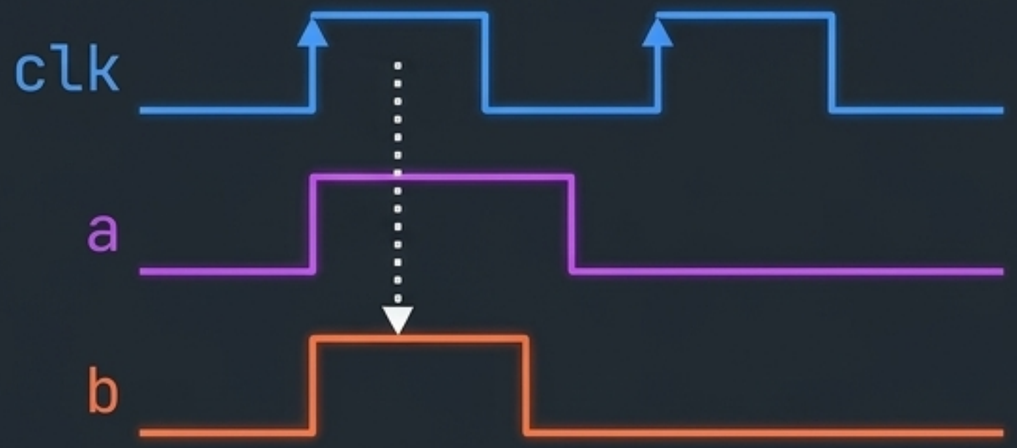


Implication is an **if-then** structure exclusively for properties. It prevents false failures.

NotebookLM

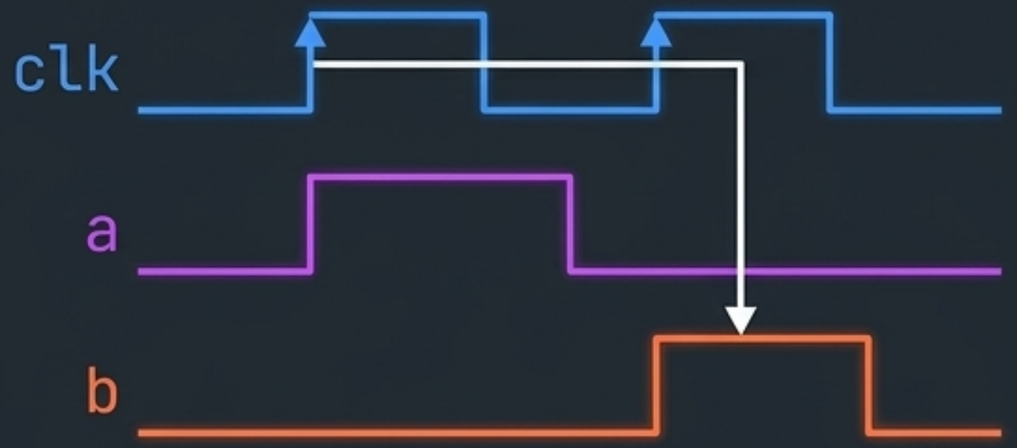
The Implication Matrix

```
@(posedge clk) a |-> b;
```



Rule: Consequent evaluates in the SAME clock cycle.

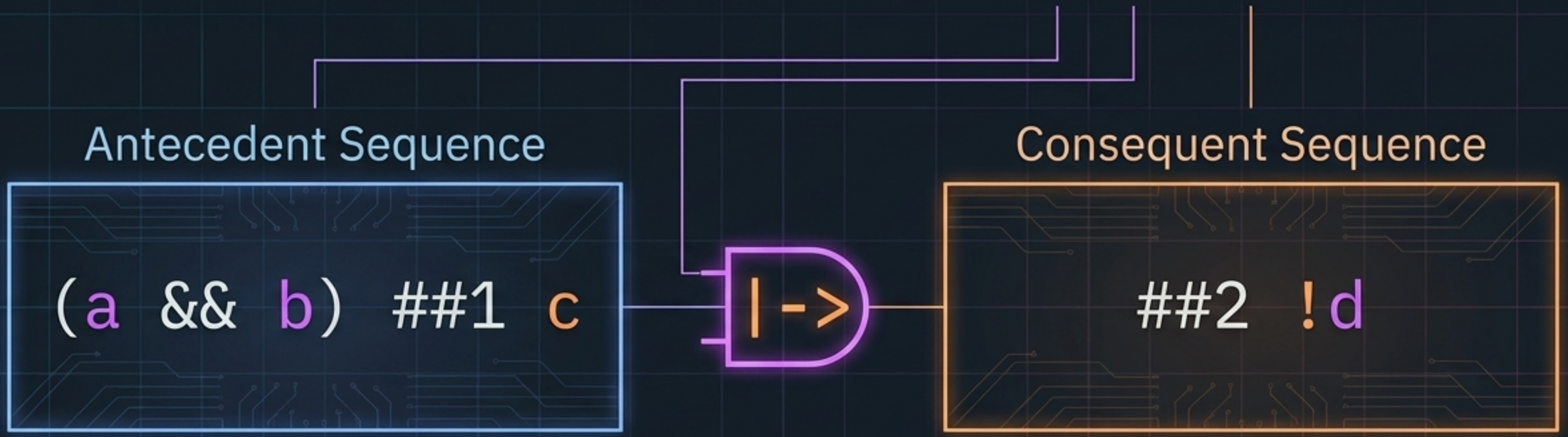
```
@(posedge clk) a |=> b;
```



Rule: Consequent evaluates in the NEXT clock cycle.

NotebookLM

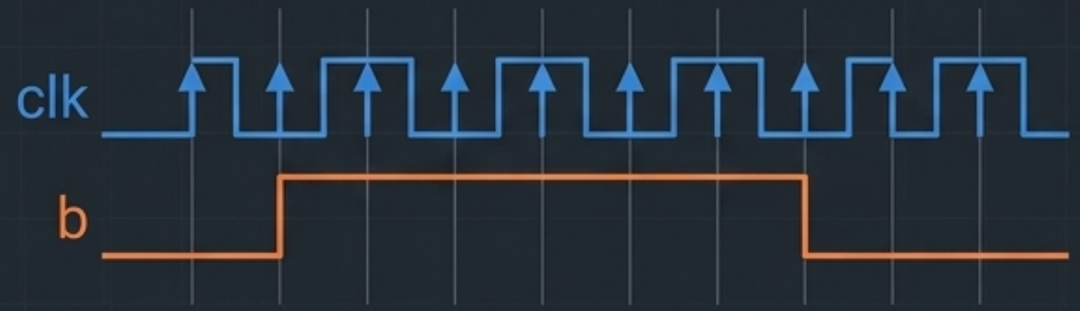
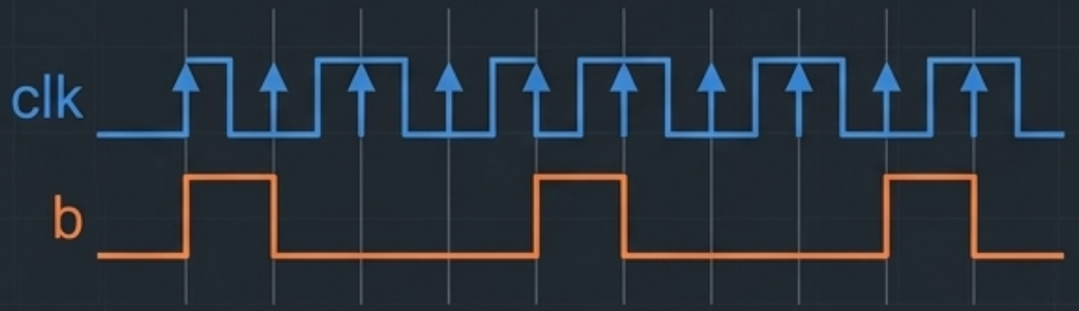
```
@(posedge clk) seq_1 |-> seq_2;
```



If **a** and **b** are true, and **c** is true one cycle later... → **ONLY THEN** → wait 2 cycles and check that **d** is low.

NotebookLM

The Repetition Matrix

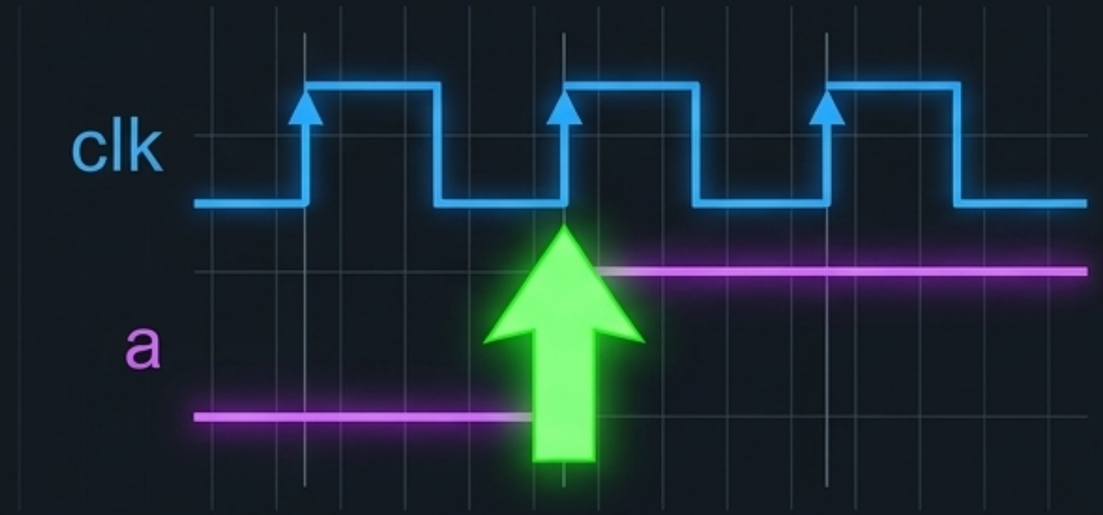
Consecutive ([*n])	Go-To ([->n])
b[*3]	b[->3]
	
Rule: Match continuously for the specified number of clocks.	**Rule:** Match the specified number of times, not necessarily on continuous clock cycles.
Equivalence: `a ##1 b[*3] == a ##1 b ##1 b ##1 b`	

NotebookLM

Essential Edge-Detection System Functions

$rose

`$rose(a)`



Checks that the signal transitioned to a value of 1 on the positive edge of the clock.

$fell

`$fell(a)`



Checks that the signal transitioned to a value of 0 on the positive edge of the clock.

NotebookLM

The Master Equation: Synthesizing SVA

Clock Definition
Best Practice: define inside property, keep sequences independent.

Sequence Delay
Ack followed 1 cycle later by Done.

```
property handshake_check;
  @(posedge clk) $rose(req) |=> (ack ##1 done)[*3];
endproperty

assert property (handshake_check);
```

Transition Antecedent
Only checks when Request goes high.

Non-overlapped Implication
Begin checking the consequent on the NEXT clock cycle.

Consecutive Repetition
The Ack -> Done sequence must happen 3 times continuously.

NotebookLM

SVA Cheat Sheet: Essential Syntax & Functions

Temporal Operators

`##n`	Exact Delay
`##[min:max]`	Bounded Window
`##[1:$]`	Eventuality (Infinite)

Implications

`|->`	Overlapped (Same cycle check)
`|=>`	Non-overlapped (Next cycle check)

Repetitions

`[*n]`	Consecutive match
`[->n]`	Go-to intermittent match

System Functions

`$rose(x)`	0 to 1 positive transition
`$fell(x)`	1 to 0 negative transition

Severity Levels

`$fatal` : Halts simulation | `$error` : Run-time error | `$warning` : Suppressible | `$info` : Informative

The SVA Language Architecture: Sequence events over time. Gate with implications. Assert properties.

NotebookLM